

Eficiência Energética em Software: Impacto da Qualidade do Código e da Linguagem de Programação

Aluno: Leandro Cardoso

Instituição: Univassouras

Curso: Engenharia de Software

Disciplina: Introdução a Eficiência Energética

Professor: Dr. Sergio

Data: Maio de 2026

Resumo

A eficiência energética tornou-se um tema relevante na engenharia de software devido ao crescimento do consumo energético de sistemas computacionais, datacenters e dispositivos embarcados. Este trabalho analisa o impacto da qualidade da implementação de algoritmos e da escolha da linguagem de programação sobre o desempenho computacional e, conseqüentemente, sobre o consumo energético. Para isso, foram desenvolvidas quatro implementações de um mesmo problema computacional: versões otimizada e não otimizada em C++ e Python. Os resultados demonstram que a qualidade do algoritmo possui impacto significativamente maior que a própria linguagem utilizada, reduzindo milhares de vezes a quantidade de instruções executadas, ciclos de CPU consumidos e tempo total de processamento.

Palavras-chave: eficiência energética, engenharia de software, desempenho, otimização, C++, Python.

1. Introdução

O crescimento da demanda por processamento computacional tem aumentado a preocupação com o consumo energético de aplicações de software. Embora a evolução do hardware contribua para ganhos de eficiência, a forma como um

software é desenvolvido continua sendo um fator determinante para o uso eficiente dos recursos computacionais.

Algoritmos mal projetados podem executar bilhões de operações desnecessárias, aumentando o tempo de processamento, a utilização da CPU e o consumo de energia. Em contrapartida, algoritmos eficientes reduzem significativamente a quantidade de trabalho executado pelo processador.

Este trabalho busca demonstrar experimentalmente o impacto da qualidade da implementação e da linguagem de programação na eficiência computacional e energética.

2. Objetivos

2.1 Objetivo Geral

Avaliar o impacto da qualidade do código e da linguagem de programação sobre o desempenho computacional e a eficiência energética.

2.2 Objetivos Específicos

- Comparar implementações otimizadas e não otimizadas.
- Comparar o desempenho entre C++ e Python.
- Quantificar ciclos de CPU e instruções executadas.
- Relacionar desempenho computacional ao consumo energético.

3. Fundamentação Teórica

A eficiência energética de um software está diretamente relacionada à quantidade de recursos computacionais necessários para sua execução.

O consumo de energia pode ser representado pela Equação 1:

Equação 1

$$E = P \times t$$

Onde:

- E representa a energia consumida;
- P representa a potência média consumida;
- t representa o tempo de execução.

Dessa forma, mesmo que um processador opere com potência semelhante, uma aplicação que finalize sua execução em menos tempo tende a consumir menos energia.

Além disso, a complexidade algorítmica exerce forte influência sobre a quantidade de operações realizadas. Algoritmos com complexidade elevada executam muito mais instruções e consomem mais ciclos de CPU, aumentando o tempo necessário para concluir a tarefa.

4. Metodologia

4.1 Problema Avaliado

Foi implementado um algoritmo responsável por:

1. Identificar números primos;
2. Calcular o quadrado de cada número primo;
3. Somar todos os quadrados encontrados.

Foram desenvolvidas quatro versões:

Implementação	Linguagem	Qualidade
bad_cpp	C++	Não otimizada
good_cpp	C++	Otimizada
bad_python	Python	Não otimizada
good_python	Python	Otimizada

4.2 Versão Não Otimizada

A versão não otimizada realiza:

- Verificação de divisores até $n-1$;
- Continuação da busca mesmo após encontrar um divisor;
- Grande quantidade de operações redundantes.

Essa abordagem possui complexidade significativamente maior.

4.3 Versão Otimizada

A versão otimizada aplica:

- Verificação apenas até $\sqrt{n}$;

- Encerramento imediato ao encontrar divisor;
- Eliminação de verificações desnecessárias para números pares.

Essas melhorias reduzem drasticamente a quantidade de operações executadas.

4.4 Ambiente de Testes

Os experimentos foram executados em ambiente Linux utilizando:

- Sistema Operacional: Arch Linux
- Compilador C++: GCC
- Interpretador Python: Python 3
- Ferramenta de medição de tempo: /usr/bin/time
- Ferramenta de análise de hardware: perf

Cada implementação foi executada cinco vezes, sendo considerados os valores médios obtidos.

5. Resultados

5.1 Tempo de Execução

Implementação	Tempo Médio (s)
C++ Não Otimizado	7,234
C++ Otimizado	< 0,001
Python Não Otimizado	130,498
Python Otimizado	0,040

Observa-se uma redução extremamente significativa no tempo de execução após a aplicação das otimizações.

5.2 Ciclos de CPU

Implementação	Ciclos
C++ Não Otimizado	35.046.336.205
C++ Otimizado	16.061.319
Python Não Otimizado	615.171.562.658
Python Otimizado	179.724.043

5.3 Instruções Executadas

Implementação	Instruções
C++ Não Otimizado	50.012.496.373
C++ Otimizado	22.356.679
Python Não Otimizado	2.751.308.340.059
Python Otimizado	717.034.052

5.4 IPC (Instructions Per Cycle)

Implementação	IPC
C++ Não Otimizado	1,427
C++ Otimizado	1,392
Python Não Otimizado	4,472
Python Otimizado	3,990

6. Discussão

Os resultados demonstram que a otimização do algoritmo produziu ganhos expressivamente superiores aos obtidos apenas pela escolha da linguagem.

Comparando as versões em Python:

- O tempo foi reduzido de 130,498 segundos para 0,040 segundos.
- Houve redução superior a três mil vezes na quantidade de ciclos de CPU.
- Houve redução superior a três mil vezes na quantidade de instruções executadas.

Comparando as versões em C++:

- O tempo foi reduzido de 7,234 segundos para menos de 1 milissegundo.
- A quantidade de ciclos foi reduzida em aproximadamente 2.182 vezes.
- A quantidade de instruções foi reduzida em aproximadamente 2.237 vezes.

Um resultado particularmente relevante foi observado ao comparar o C++ não otimizado com o Python otimizado. Apesar de Python ser tradicionalmente considerado mais lento que C++, a implementação otimizada em Python executou aproximadamente 181 vezes mais rápido que a implementação não otimizada em C++.

Esse resultado evidencia que a qualidade do algoritmo possui influência maior que a própria linguagem de programação utilizada.

Sob a perspectiva energética, a redução da quantidade de instruções executadas e do tempo de processamento implica menor utilização dos recursos computacionais. Como o consumo de energia é proporcional ao tempo durante o qual o hardware permanece processando uma tarefa, aplicações mais eficientes tendem a consumir significativamente menos energia.

7. Conclusão

Os resultados obtidos confirmam que a qualidade da implementação possui impacto decisivo sobre o desempenho computacional e a eficiência energética de aplicações.

As otimizações aplicadas reduziram drasticamente a quantidade de instruções executadas, ciclos de CPU utilizados e tempo total de processamento. Em alguns cenários, os ganhos ultrapassaram milhares de vezes em relação às versões não otimizadas.

Embora a escolha da linguagem influencie o desempenho, os experimentos demonstraram que algoritmos eficientes podem superar amplamente implementações inadequadas mesmo quando desenvolvidas em linguagens tradicionalmente mais rápidas.

Dessa forma, conclui-se que a adoção de boas práticas de desenvolvimento e a escolha de algoritmos adequados representam estratégias fundamentais para a construção de softwares energeticamente eficientes, contribuindo para a redução do consumo de recursos computacionais e dos custos operacionais associados.

Referências

AHO, Alfred V.; HOPCROFT, John E.; ULLMAN, Jeffrey D. *Data Structures and Algorithms*. Reading: Addison-Wesley, 1983.

CORMEN, Thomas H. et al. *Introduction to Algorithms*. 4. ed. Cambridge: MIT Press, 2022.

MEYERS, Scott. *Effective Modern C++*. Sebastopol: O'Reilly Media, 2014.

LUTZ, Mark. *Learning Python*. 5. ed. Sebastopol: O'Reilly Media, 2013.

PATTERSON, David A.; HENNESSY, John L. *Computer Organization and Design*. 5. ed. Burlington: Morgan Kaufmann, 2013.